Ders 1 Genişletilmiş Notu: Büyük Resim; Ölçme, Örnekleme, Nicemleme ve Sayıların Gösterimi

Ders 1 Genişletilmiş Notu: Büyük Resim; Ölçme, Örnekleme, Nicemleme ve Sayıların Gösterimi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Adı ve Kapsamı
- 2. Geçiş Dersi Olarak Sinyaller ve Sistemler
- 3. Dersten Beklenenler, Araçlar ve Kaynaklar
- 4. Dersin Haritası
- 5. Bilişim: Veri, Bilgi, Bilgi Birikimi ve Bilgelik
- 6. Sistem Kavramı
- 7. Sinyal Kavramı
- 8. Ölçme Sisteminin Adımları
- 9. Analog Sinyalden Sayısal Sinyale
- 10. Analog-Sayısal Dönüşümün Dört Adımı ve Filtreleme
- 11. Örnekleme
- 12. Nicemleme
- 13. Sayısallaştırma ve Geri Oluşturma Zinciri
- 14. Verinin ve Sayıların Bilgisayarda Gösterimi

1. Dersin Adı ve Kapsamı

1.1 Neden “Bilgisayar Mühendisleri için”?

Sinyaller ve sistemler dersi kontrol, elektronik, elektrik ve makine gibi birçok bölümde okutulur. İçinde sinyal ve sistem geçen her alan bu konularla ilgilenir. Fakat bu alanlardaki bakış, elektronikçi kadar elektronik ya da kontrolcü kadar kontrol bilgisi isteyebilir. Bu ders konunun bilgisayar mühendisine bakan tarafıyla ilgilenir. Dersin adındaki “bilgisayar mühendisleri için” ifadesi buradan gelir.

1.2 Konu Akışı

Derse sinyallerin ve sistemlerin matematiksel tanımıyla başlanacak. Ardından temel doğal sinyaller olan sinüzoidler, yani sinüs ve kosinüs fonksiyonları ve bunların farklı gösterimleri incelenecek. Dersin büyük kısmı matematiksel işlemlerden oluşur. Sonra sinyaller frekans düzleminde gösterilecek (spectrum representation). Örnekleme ve örtüşme (sampling, aliasing) konusu normalde bu dersin asıl konusu değildir. Bir kısmı bu derslerde anlatılacak, gerekirse dönem sonunda tekrar ele alınacak. Ayrıntısı daha çok sayısal işaret işleme dersindedir.

Bu giriş bölümlerinden sonra ders ikiye ayrılır: sürekli zaman (continuous time) ve ayrık zaman (discrete time). Sinyal ve sistemler üzerindeki işlemler her iki tarafta ayrı ayrı görülecek.

- **Doğrusal zamanla değişmeyen sistemler:** Sürekli zamanda konvolüsyon integraliyle, ayrık zamanda konvolüsyon toplamıyla incelenir. Hoca bunu şöyle özetledi: “continuous” denince aklımıza integral, “discrete” denince toplam sembolü gelmeli.

- **Denklemler:** İntegral işin içine girince türev de girer. Sürekli sistemlerde diferansiyel denklemler (differential equation), ayrık sistemlerde fark denklemleri (difference equation) vardır. Fark denklemleri ayrık matematikte, algoritma analizinde de kısmen görülür.
- **Dönüşümler:** Sürekli tarafta Laplace ve Fourier dönüşümleri, ayrık tarafta z dönüşümü ve ayrık Fourier dönüşümü kullanılır.

Diferansiyel denklem çözmekle ilgilenilmeyecek. Hoca bunu makinecilerin ve elektronikçilerin tarafı olarak gösterdi. Bilgisayar mühendisine daha yakın olan fark denklemleri üzerinde durulacak. Çünkü bilgisayar mühendisi olarak ayrık verilerle çalışırız.

Dersin 11 bölümünden örnekleme ve örtüşme bölümü çıkarılınca fiilen 10 bölüm kalır.

1.3 Düzlemler Arası Geçiş Neden Yapılır?

Bir dönüşüm, sinyali bir düzlemden başka bir düzleme taşır. Örneğin t zaman düzleminden frekans düzlemine, s düzlemine ya da z düzlemine geçilebilir. Böylece aynı sinyal ya da sistem farklı bakış açılarından incelenir.

Hocanın örneği ses işaretiydi. Ses, zamana göre değişen bir fonksiyondur. Kaydedildiğinde bir dalga elde edilir. Bu dalganın zaman düzlemindeki görüntüsü istenen bilgiyi her zaman vermeyebilir. Örneğin işaretin frekansı hakkında yeterli bilginiz yoksa frekans düzlemine geçmeniz gerekir. Daha fazla bilgi için s düzlemine ya da başka düzlemlere de geçilebilir.

2. Geçiş Dersi Olarak Sinyaller ve Sistemler

Hoca dersi bir geçiş kapısına benzetti. Dersin öncesi ve sonrası vardır. Burada öğrenilenleri eski bilgilerle ilişkilendirmek gerekir.

- **Öncesi, sayısal dünya:** Bilgisayar bilimlerine giriş (Introduction to Computer Science) dersinde verinin bilgisayarda nasıl saklandığı ve işlendiği görülür.
- **Öncesi, analog dünya:** Devre teorisi (circuit theory) dersinde direnç, akım ve aralarındaki ilişkiler görülür.
- **Bu ders:** Analog dünyadaki problemleri sayısal dünyada çözmek için kullanılan yaklaşımların en temel olanlarını içerir.
- **Sonrası:** Sayısal işaret işleme (digital signal processing), oradan görüntü işleme (digital image processing), video işleme (digital video processing) ve ses işleme (digital sound processing) gelir.
- **Yapay zeka ile kesişim:** Bu işleme adımları bilgisayarla görme (computer vision), akıllı sistemler ve otonom robotlar gibi yapay zeka uygulamalarının ön adımıdır. Örneğin bir ses işlenecekse, önce sesin karakteristiklerini ortaya çıkarmak için sinyal işleme ve ses işleme adımları uygulanır. Sonrasında yapay zeka teknikleri kullanılır.

  Hocanın vurgusu: Bu ders öğrenciler tarafından ilk başta sevilmeyebilir. Ama gerçek anlamda mühendislik, yani tasarım yapılacaksa, hele yapay zeka ve derin öğrenme ile uğraşılacaksa buradaki bilgiler çok önemlidir.

Soru: Bu dersten başarılı olmak için devre teorisi ve bilgisayar bilimleri konusunda ne kadar bilgi gerekir? (Hoca bu soruyu geçmiş dönemde gelen bir sorudan yola çıkarak sordu.) **Cevap:** Devre teorisindeki temel kavramları bilmek yeterlidir. Devrelerle orada olduğu kadar uğraşılmaz. Burada doğrudan sistem tasarımı yoktur, sistemlerin özelliklerini öğrenmek hedeflenir. Sayısal sinyal işleme gibi derslerde devre teorisine biraz daha ihtiyaç olur.

Hoca, sinyal işlemenin yalnız sayısal olmadığını da ekledi. Analog sinyal işleme ve analog görüntü işleme de yapılabilir. Önceden fotoğrafçıların görüntü üzerinde yaptığı rötuş buna örnektir. Elektronik bölümünde okunan analog elektronik, analog devre tasarımı ve analizi dersleri bu tarafla ilgilidir.

3. Dersten Beklenenler, Araçlar ve Kaynaklar

3.1 Sinyali Matematiksel Tanımlamak

Bu dersten beklenen ilk şey, verilen bir sinyali matematiksel olarak, özellikle bir fonksiyon cinsinden yazabilmektir. Hocanın ilk örneği şuydu:

$$y(t) = A \cdot x(t)$$

Burada $x(t)$ bir sinyaldir, $y(t)$ de bir sinyaldir. $x(t)$'den $y(t)$'yi oluşturan fonksiyon ise bir sistemdir. Neden sistem deriz? x kendi içinde zamana göre değişir, y de zamana göre değişir. Ama y aynı zamanda x'in değerlerine göre değişir. Bu örnekte zamana göre elde edilen x değerleri sabit bir katsayıyla çarpılır ve yine zamana göre değişen yeni y değerleri elde edilir.

3.2 Programlama Araçları

Dersin kod yazma tarafı C'deki gibi algoritmayı sıfırdan kodlamak değildir. Algoritmaları içinde barındıran dillerin hazır fonksiyonları kullanılır. Hoca bu dilleri, sinyalci, elektronikçi ve kontrolcülerin çoktan aza kullanımına göre MATLAB, Octave ve Scilab olarak sıraladı. Python ve R da kullanılabilir. Hocanın önerisi MATLAB ya da Python'dan birini seçmektir. R'ı da veri bilimindeki yaygınlığı nedeniyle seçenek olarak andı.

Bu dillerin tercih edilme sebebi sonuca kolay ulaşmaktır. Grafik çizmek ve Fourier dönüşümü gibi hazır işlemleri yapmak isteriz. Bunlar C'de sıfırdan yazılabilir ama kolay değildir. Amaç Fourier dönüşümü kodu yazmak değil, ondan çıkan sonuçları yorumlayabilmektir. Yine de bazı durumlarda algoritmayı kendiniz yazıp hazır fonksiyonla karşılaştırmak öğreticidir. Hocanın örneği konvolüsyondu: Hazır kodu vardır, algoritması da basittir, ikisinin sonuçları karşılaştırılabilir.

- **MATLAB:** Kendi programlama dili olan bir paket programdır. Döngü, dizi ve değişken tanımı vardır. Komut penceresine (command window) yazılan kod satır satır yorumlanır ve sonuç hemen görülür. Çalışılan dizinin dosyaları ve tanımlanan değişkenler ayrı panellerde izlenir. Örneğin `x` değişkenine 5 atandığında değişken panelde otomatik görünür. Yardım dokümantasyonu çok iyidir, her fonksiyon tanımı ve örneğiyle incelenebilir. Üniversitenin lisansı vardır, ama kurulumu yaklaşık 7 GB tutar ve her fonksiyonu kullanılmayacaktır. Bu yüzden hoca Online MATLAB'ı önerdi. Açık kaynak değildir.
- **Octave ve Scilab:** MATLAB'a çok benzeyen açık kaynaklı programlardır. Octave için MATLAB'ın açık kaynak hali denir. MATLAB kodları Octave'da doğrudan çalıştırılabilir, ikisinin dosya uzantısı da `.m`'dir. Özellikle grafik çizimleri bu derste işe yarar.
- **R:** RStudio adlı geliştirme ortamıyla kullanılır, dil ayrıca kurulur. Arayüzü MATLAB'a benzer, sözdizimi biraz farklıdır.
- **Python:** Diğerlerinden öne çıkar, çünkü tam bir programlama dilidir ve sinyal işlemede de çok kullanışlıdır. Açık kaynaktır ve ileride başka derslerde de kullanılacaktır.
- **Anaconda:** Python dilini ve kolay programlama için gereken özellikleri içinde barındıran paket programdır. Hoca bunu Python için bir çeşit Visual Studio'ya benzetti. Navigator üzerinden Spyder, PyCharm, Jupyter Notebook, JupyterLab, Visual Studio Code ve RStudio başlatılabilir ya da eklenebilir. Kod yazmadan makine öğrenmesi yöntemlerini denemeye yarayan Orange aracı da buradadır.

  Hocanın vurgusu: Her iş için yeni bir dil öğrenmek marifet değildir. Önemli olan bir dili iyi kullanabilmektir. Ya Python tarafında ya MATLAB tarafında olun. Hoca Python'ı daha çok tercih ediyor.

3.3 Dersin Kazandırmak İstedikleri

Ders, programlama ve devre bilgisinin üstüne akademik bir bakış ekler. Sinyale zaman ve frekans açısından bakmak, bu düzlemlerin farkı ve aralarındaki geçiş, ayrık ve sürekli zamanın ne olduğu bu bakışın parçasıdır.

Asıl hedeflerden biri gerçek dünya problemini modelleyebilmektir. Hoca çok disiplinli tasarım projesinden bir örnek verdi. Kontrol, elektronik, elektrik ve bilgisayar öğrencileri bir seranın sıcaklık sistemini modellemişti. Seranın büyüklüğü ve sıcaklığı etkileyen birçok değişken bu

modele girer. Kontrol tarafındaki öğrenciler bu parametreleri dikkate alarak sıcaklığı kontrol edecek bir denetleyici tasarlamıştı. Bu dersin faydası böyle bir sistemin matematiksel modelini kurabilmektir. Model kurulduktan sonra sonuçları yorumlamak gerekir. Kararlılık (stability) bu yorumlardan biridir.

Pratik hedef, öğrenilenleri bir dil üzerinde uygulayabilmektir. Aktarılabilir kazanım olarak hoca iki şeyi saydı. Birincisi, yapay zeka, makine öğrenmesi ve derin öğrenmede kullanılan yöntemlerin ön adımları bu dersin içeriğinden geçer. İkincisi, yapılan bilimsel bir çalışmanın nasıl sunulacağı, nasıl makale yazılacağı ve sunumda hangi anahtar noktaların kullanılacağıdır.

3.4 Kaynak Kitaplar

- ***Signal Processing First*:** Dersin ana kitabıdır. İngilizcesi ağır değildir. Anlaşılmayan kavramları pekiştirmek ve örnek soru çözmek için kullanılabilir.
- ***Signals and Systems* (Oppenheim, MIT):** Bu alandaki kitapların çoğunun esinlendiği temel kaynaktır. Yazarının ders video kayıtları da YouTube'da bulunabilir.
- ***Signals and Systems for Dummies*:** Konuları kolay bir dille, bilinenden bilinmeyene doğru anlatır.
- **Çevrimiçi bir e-kitap:** Slaytta adresi gösterildi.
- **Adams'ın kitabı ve slaytları:** Kanada'da ders veren bir hocanın ücretsiz indirilebilen kitabı, slaytları ve YouTube dersleridir. Hoca bu kaynağa zaman zaman başvuracağını ve sınıflandırmalarının çok iyi olduğunu söyledi.

4. Dersin Haritası

Hoca, kitaptan alınan bir şema üzerinden büyük resmi yeniden özetledi. Önce sinyal ve sistemlerin temelleri görülür. Sonra iki kol başlar: sürekli zaman ve ayrık zaman. Sürekli zamandan ayrık zamana geçmek için yapılan işlem örneklemedir (sampling).

Sırasıyla şu basamaklar izlenir:

- **Temel kavramlar:** Sürekli zaman, sürekli sistem, ayrık zaman ve ayrık sistem nedir?
- **Konvolüsyon:** Sürekli zamanda konvolüsyon integrali, ayrık zamanda konvolüsyon toplamı.
- **Zaman düzleminde denklemler:** Diferansiyel denklemler ve fark denklemleri.
- **Frekans düzlemi, sürekli taraf:** Fourier serileri ve Fourier dönüşümü. İki ayrı işlem vardır, çünkü işlem sinyalin periyodik olup olmamasına göre değişir.
- **Frekans düzlemi, ayrık taraf:** Ayrık zamanlı Fourier dönüşümü (DTFT), ayrık Fourier dönüşümü (DFT) ve hızlı Fourier dönüşümü (FFT).
- s **ve** z **düzlemleri:** Sürekli zamanda s düzlemi, yani Laplace dönüşümü ve özellikleri. Ayrık zamanda z düzlemi, yani z dönüşümü ve özellikleri.

Bütün bu öğrenilenlerle bir sinyal ya da sistem üzerinden bilgi çıkarılıp çıkarılamayacağı, yorum yapılıp yapılamayacağı konuşulacak.

> **Hocanın vurgusu:** DTFT başka bir şeydir, DFT başka bir şeydir. Bu farkı bilmek hem genel kültür hem de ne yaptığınızı anlamak için önemlidir.
>
> **Ek bilgi (derste anlatılmadı):** DTFT'nin sonucu büyük olasılıkla frekansın sürekli bir fonksiyonudur. DFT ise sonlu sayıda frekans noktası üretir. FFT ayrı bir dönüşüm değil, DFT'yi hızlı hesaplayan algoritmadır. Bu ayrımın ayrıntısı dersin ilerleyen bölümlerine kalıyor.

5. Bilişim: Veri, Bilgi, Bilgi Birikimi ve Bilgelik

Birinci bölümün slaytları, bilgisayar bilimlerine giriş ve yapısal programlama derslerinde anlatılanları toparlamak için hazırlanmıştır. Bu bilgiler dersin ön gereksinimlerinden sayılır.

> **Hocanın vurgusu:** Buradan doğrudan soru sorulması düşünülmüyor, ama dolaylı sorular gelebilir. Örneğin n bit ile kaç farklı durum temsil edildiğini, yani 2^n'i artık bilmemek "biraz anormal" sayılır.

Bilişim (informatics), geniş anlamda bilginin oluşturulması, saklanması, bulunması, işlenmesi ve paylaşılmasıyla ilgilidir. Dilbilim gibi farklı bilim dallarıyla da birlikte çalışılabilir. Kavramın etimolojisi slaytta yer alıyor, hoca bunun üzerinde durmadı.

5.1 Dört Kavram

- **Veri (data):** İşlenmemiş ham değerlerdir. Olaylar, gerçekler ve rakamlar böyledir. İçinde yorum ya da analiz yoktur. Örnek: Ham petrolün varil fiyatının şu kadar olması.
- **Bilgi (information):** Veriden yorum yapılarak çıkan sonuçtur. Örnek: Fiyatın şu değerden şu değere arttığının söylenmesi.
- **Bilgi birikimi (knowledge):** Bilgi, tecrübe ve öngörülerin bir araya gelmesidir. İleride çıkarım ve tahmin yapmayı, analiz ve karar vermeyi sağlayan bir bilgi bankası gibidir. Hoca bu noktada "insight" kavramını da açıkladı: Bir şeyin içini görmek, konuyu derinlemesine bilmek.
- **Bilgelik (wisdom):** Bunların da ötesindedir. Örnek: "Varil fiyatı şu kadar artarsa, litre fiyatının yıl başına kadar şu kadar artması muhtemeldir" gibi geleceğe dönük bir öngörü.

Hayat bizi bu tahminlere zorlar. Sosyal olaylarda da, içinde bulunulan hastalık dönemi ya da yangın dönemleri gibi durumlarda da eldeki birikimden geleceğe dönük tahmin yapmak gerekir.

Hoca aynı zinciri bu dersin diliyle kurdu. Zamana bağlı $x(t)$ sinyallerimiz toplanır, onlardan bilgi elde edilir. Bilgi işlenerek bilgi birikimine, oradan geleceğe dönük tahmine geçilir. Slayttaki iş dünyası örneği de aynıdır:

- Ham satış fiyatları veridir.
- Satış yöneticisinin bunlardan hazırladığı satış raporu bilgidir.
- Fiyatları artırma, azaltma ya da başka bir aksiyon alma kararı bilgi birikimine dayanır.

5.2 Veri Toplamak Neden Pahalıdır?

Temiz ve yapılandırılmış veri toplamak masraflıdır. Pahalılık yalnız para değil, iş gücü demektir. Bir yapay zeka işi yapılacaksa önünüze birçok ön problem çıkar. Bunların en önemlilerinden biri verinin yanlış toplanmasıdır. Bu yüzden işin ne için yapılacağını iyi bilmek gerekir.

Bir şeyi kontrol etmek ve iyileştirmek için, onun performansını gerçekten etkileyen faktörleri bulup bunlara yönelik veri toplamak gerekir. Hocanın örneği: Dolar üzerinde tahmin yapacaksanız, doların hareketini etkileyen parametreleri belirleyip buna göre veri toplamalısınız. Veri düzenli ve güvenli biçimde toplanmalı, ihtiyaç duyulan bilgiye dönüştürülebilmelidir.

5.3 Faydalı Verinin Özellikleri

- **Amaca uygun:** Alakasız verilerle uğraşılmamalıdır.
- **Tam:** Eksik veriyle çalışmak çoğu zaman yanlışa götürür.
- **Doğru (accuracy):** Bilimsel yayınlarda yanlış veri toplanması ret sebebidir. Proje kabul edilmeyebilir, kabul edildiyse sonradan iptal edilebilir.
- **Zamanında:** Zamanlama önemlidir.
- **Doğru formatta:** Örneğin bir hesap tablosunda analiz edilebilir, bir bilgisayar sistemine girilebilir olmalıdır.
- **Makul maliyetli:** Hoca "attığınız taş ürküttüğünüz kuşa değsin" deyişiyle anlattı. Toplanacak veri uygulanabilir (feasible) olmalıdır.

Aynı şartlar bilgi için de geçerlidir.

5.4 Bilgi ile Bilgi Birikimi Karışıklığı

Bu iki kavram sık karıştırılır. Hoca felsefesine girmeden iki tür bilgi birikimini andı. Açık, biçimsel ve genel kabul görebilen türü (explicit, formal) politika geliştirmede ve geleceğe dönük kararlarda kullanılır. Sezgisel, bilinçaltı, örtük ya da gizli türü (instinctive, subconscious, tacit, hidden) ise hocanın benzetmesiyle bir şey verilirken arka planda taşınan gizli bir mesaj gibidir. Her ikisi de organizasyonlar için önemlidir ve bilginin kullanımı bunlara geçişin anahtarıdır.

6. Sistem Kavramı

6.1 Tanım ve Örnekler

Sistem, bir hedefi yerine getirmek için bir araya getirilmiş, tümleşik elemanların kümesidir. Hocanın ilk örneği otomobilin fren sistemiydi. Pedal bir elemandır, ardından başka elemanlar gelir, aralarında bir ilişki vardır ve hepsi bir hedef için bir araya getirilmiştir.

Tanım mühendislik alanına göre değişir:

- **Yazılım mühendisi için:** Elemanlar bilgisayar programlarıdır. Örnek: Öğrenci yönetim sistemi.
- **Elektrik mühendisi için:** Elemanlar bir devre üzerinde belli bir hedef için birleştirilir. Örnek: Elektrik devre sistemi.

Kişiden kişiye değiştiği için "belli bir hedefe yönelik oluşturulmuş tümleşik elemanların kümesi" uygun bir tanımdır. Sistem aslında parçaların bir araya getirilmesidir (assembly). Parçalar bir şekilde birbirine bağlanır ve bağlanma şekli alandan alana değişir. Birbirlerini etkiler, birbirlerinden etkilenirler. Ortaya çıkan şey bir görevi yerine getirir ve buna bir ad veririz, örneğin fren sistemi. Slayttaki bir başka tanıma göre kaynakların ele alınmasını, işlenmesini ya da yönetilmesini içeren her düzenleme sistem olarak gösterilebilir: "any arrangement which involves the handling, processing or manipulation of resources of whatever type can be represented as a system."

Sistem büyük ve karmaşık olabilir. Trafik kontrol sistemi ve haberleşme sistemi buna örnektir. Haberleşmede size en yakın eleman telefonunuzdur. Ama sistem yalnız telefondan oluşmaz: Uydu, GPS, karşıdaki cihaz, aradaki geçişler ve işlemler de sistemin içindedir. Küçük bir cihaz da sistem olabilir. Cep hesap makinesi, alarm, girişlerde ateş ölçen ve sensörle dezenfektan damlatan cihaz bunlara örnektir.

6.2 Giriş, İşlem, Çıkış ve Geri Besleme

Sistemin temel yapısı bilgisayar gibidir: girişler (input), işlemler (process) ve çıkışlar (output). Sisteme geri besleme (feedback) eklenirse geri beslemeli sistem ya da kontrol sistemi elde edilir.

Elde edilen çıkış istenen (desired) ya da gerçek çıkışla karşılaştırılır. Bu karşılaştırmanın sonucu geri beslemedir. Ona göre sistemin işlemlerinde ya da başka bir bileşeninde güncelleme yapılabilir. Sistem böylece geleceğe dönük tahmin yapabilecek hale getirilir. Hoca buna "eğitim" dendiğini söyledi.

6.3 Model

Sistemler modellerle ifade edilir. Model, matematikteki fonksiyonlara benzer. Her zaman fonksiyon olmaz, bağıntı da diyebiliriz. Sayısal analiz dersinde görülen regresyon ve interpolasyon da böyledir: Toplanan verilerden bir fonksiyon üretilir, yeni gelen durum o fonksiyon üzerinde tahmin edilir. Model kurulunca elemanlar arasındaki ilişki bulunmuş olur ve geleceğe dönük tahmin kolaylaşır.

Hocanın örneği yol-hız bağıntısıydı:

$$x = v \cdot t$$

Hız ile zamanın çarpımı yolu verir. Bu bağıntıyla belli bir zamanda ne kadar yol alınacağı hesaplanabilir. Bu bir modeldir. Sabit hızla gidilmiyorsa hız da zamana göre değişir, model zamana bağlı hale gelir. Bu derste bu tarz sistemler analiz edilecek.

6.4 Bilgi Sistemleri ve Sayısal Sistemler

Bilgi sistemlerinin dört işlevi kaydetme, taşıma, düzenleme ve işlemedir. Bir bilgi sistemini gerçekleyen bileşenler şunlardır:

- Donanım (hardware): bilgisayarlar, ağ donanımları ve diğer fiziksel araçlar.

- Yazılım (software): donanımı yönetmek ve ona komut vermek için gerekir.
- İnsanlar.
- Kurallar.
- Veriler ve veri tabanları.

Sayısal sistemde girişler, iç durumlar (state) ve çıkışlar ayrık bilgidir. Yani sayısal sistem ayrıklaştırmaya ihtiyaç duyar. Sayısal bilgisayar bunun bilinen örneğidir: Veriler bellekte (memory) geçici ya da kalıcı saklanır, işlem birimi (CPU) içinde kontrol birimi gibi parçalar vardır, veriler veri yollarıyla bellek ve işlem birimi arasında taşınır. Sistem ayrıca giriş, çıkış ve yardımcı cihazlar içerir ve işlem eş zamanlı (senkron) ya da eş zamansız (asenkron) olabilir.

Hoca komut döngüsünün adımlarını sordu ve kendisi hatırlattı. Bunlara pipeline da demiştik:

- **Getir (fetch instruction):** Komut bellekten getirilir.
- **Çöz (decode instruction):** Komut çözülür.
- **Yürüt (execute instruction):** Komut ne gerektiriyorsa o yapılır.
- **Geri yaz (write back):** Gerekirse sonuç geri yazılır.

7. Sinyal Kavramı

Hoca sinyalden ne anladığımızı sordu. Günlük dilde “sinyal geldi” deriz.

Soru: Sinyal kavramından ne anlıyorsunuz? **Cevap:** Sınıftan “uyarı” cevabı geldi. Hoca bunu aslında bir bilginin gönderilmesi olarak yorumladı: Bir ses ya da bir değişiklik bilgisi. Bunlar kavramın içindedir, ama sinyal daha geniştir.

Sinyal, fiziksel bir büyüklüğü (physical quantity) gösteren değişkendir. Yani bilgi saklayan bir değişkendir. Sayısal bir sistem için sinyal ayrık verilerden oluşur.

En yaygın sayısal sistem ikili (binary) sistemdir, çünkü 0 ve 5 volt ya da 0 ve 12 volt ile bütün veriler saklanabilir. İkili değerler farklı sembollerle gösterilir: 0 ve 1, yanlış ve doğru (false, true), düşük ve yüksek (low, high), kapalı ve açık (off, on). Örneğin 0 volta düşük, 5 volta yüksek denir. Normalde düşük kapalıya, yüksek açığa karşılık gelir. Slaytta bu ikisinin yeri farklı yazılmıştı. Hoca bunun değiştirilebilir bir atama olduğunu da söyledi. Bu ikili değerlerle fiziksel büyüklükleri saklarız.

8. Ölçme Sisteminin Adımları

8.1 Sıcaklık Örneği

Hoca, bir ortamın sıcaklığını ölçmek üzerinden konuyu açtı. Bir öğrenciye bulunduğu şehirdeki sıcaklığı sordu. Öğrenci bir internet sayfasından baktı ve 38 derece dedi. Hocanın vurguladığı nokta şuydu: Sayfada öyle yazdığı için sıcaklık öyle değildir, sıcaklık öyle olduğu için sayfada öyle yazar. Sayfa bilgiyi başka bir yerden, örneğin meteorolojiden almıştır. Peki meteoroloji nereden biliyor? Bunun için bir ölçme sistemi geliştirmek gerekir.

Sıcaklık için cıvalı ve alkollü termometreler, Celsius, Fahrenheit ve Kelvin gibi ölçekler vardır. Türkiye’de Celsius, Amerika’da Fahrenheit kullanılır. Bilmeyen birine oradaki değeri söyleseniz “yanıyor” der. Hoca, bizim 38 derecemizin orada daha büyük bir sayı olduğunu söyledi, hesabını hatırlamadığını belirtti.

Ders sonrası not: $38\ ^\circ\mathrm{C}$, $\frac{9}{5} \cdot 38 + 32 = 100{,}4\ ^\circ\mathrm{F}$ eder (Python ile doğrulandı).

Ölçülen değer göreceli bir bilgidir. Sıcaklık gerçek anlamda tam 38 değildir, artısı ve eksisi vardır.

8.2 Zincirin Adımları

Dönüştürücü (transducer). Dışarıdaki bir fiziksel büyüklük ya da enerji, dönüştürücü ile başka bir forma çevrilir. Dönüştürücünün fiziksel büyüklüğü elektriksel işarete çeviren özel haline sensör denir. Elektriksel büyüklük olarak gerilim (volt) ya da akım (amper) elde edilir. Sonuç göreceli bir değerdir. Dönüştürücü düzgün çalışmayabileceği için hatalar en başta buradan kaynaklanır.

Analog işleme. Sinyali güçlendirmek (amplifier) ya da algılamak (detector) gerekebilir. Sıcaklık sürekli alındığı için ölçmeye başlandığının görülmesi de gerekir. Gerektiğinde analog sinyal işleme (analog signal processing) yapılır: İçinde gürültü varsa analog olarak temizlenir, gerekiyorsa güçlendirilir. Bu kısım elektronikçinin tasarladığı devreyle yapılır.

Buraya kadar hâlâ analog dünyadayız. Elimizde "38 santigrat derece" yoktur, örneğin 38,5 volt gibi bir gerilim vardır. Hoca bu değeri sıcaklıkla uyumlu olsun diye seçti.

Örnekleme (sampling). Bilgisayar analog bilgiyi doğrudan saklayamaz, çünkü analog bilgi içinde sonsuzluk taşır. Bir saatlik ölçümde sonsuz tane sıcaklık değeri vardır, her anı kaydetmek mümkün değildir. Bu yüzden analog-sayısal dönüştürücü (Analog to Digital Converter), sinyalden belli aralıklarla örnekler alır. Böylece bellekte kaç elemanlık yer ayrılacağı bulunur: N örnek için N elemanlık yer.

Soru: Bellekte N tane yer ayırdık. Saklamak için bu işlem yeterli mi? **Cevap:** Yeterli değildir. Zaman ekseninde sonsuzluğu sonlu sayıya indirdik. Aynısını genlik ekseninde de yapmamız gerekir. Çünkü her örneğin değeri de, örneğin 38,5 ya da 39,5, sonsuz olasılık içerir. 1 ile 2 arasında bile sonsuz sayıda değer vardır.

Nicemleme (quantization). Bellekte her örnek için kaç bit ayrılacağı önemlidir. n bit seçilirse, ölçülen değerin en alt ve en üst değeri arası 2^n parçaya bölünür. Gerçek değer hangi seviyeye en yakınsa ona yuvarlanır ve o seviyenin 0-1 kodu saklanır. Kodlar 00…0'dan 11…1'e kadar gider. Genlikte yapılan bu örneklemeye nicemleme denir. Bilgisayara kaydetmek için 0 ve 1 gerekir, onları bulmak için de en alt ve en üst nokta arasını 2^n parçaya bölmek gerekir.

Soru: Yuvarlama miktarı azalırsa gerçeğe daha yakın bir değer kaydedilir. Bunun için ne yapmak gerekir? **Cevap:** Bit sayısını artırmak, örneğin 8 bit yerine 16 bit kullanmak gerekir. Hoca buna ikinci bir yol ekledi: İki örnek arasında kaçırılan bir "sürpriz" olabilir. Bunu kaçırmamak için örnek sayısı da artırılmalıdır.

> **Hocanın vurgusu:** Ölçümün kalitesini iki şey artırır: büyük N, yani örnek sayısı (number of samples) ve küçük n, yani örnek başına bit sayısı.

Saklama. Örnekleme ve nicemlemeden sonra sinyal saklanabilir ya da tamponlanabilir (buffering). Bilgisayarda saklanan şeyin C'de görülen `int dizi[n]` gibi bir diziden farkı yoktur.

8.3 CD Kalitesinde Müzik Örneği

CD kalitesinde bir müzik için saniyede 44 100 örnek alınır. Ses kartı ya da mikrofon ayarlarında görülen çözünürlük (resolution) bilgisi 2^n ile bulunur. Genellikle 16 bit görülür, yani 2^{16} farklı seviye.

Bir dakikalık sesin bellekte kapladığı yer adım adım şöyledir:

- **Verilen:** $f_s = 44\,100$ örnek/saniye, süre 60 saniye, örnek başına 16 bit.
- **Örnek sayısı:** $44\,100 \times 60 = 2\,646\,000$ örnek.
- **Bit:** $2\,646\,000 \times 16 = 42\,336\,000$ bit.
- **Bayt:** $42\,336\,000/8 = 5\,292\,000$ bayt.
- **Kilobayt:** $5\,292\,000/1024 \approx 5168$ KB.
- **Megabayt:** $5168/1024 \approx 5{,}05$ MB.

  Hocanın vurgusu: Bu hesapları biliyor olmanız lazım.

  Ders sonrası not: Sonuç Python ile doğrulandı. Hoca kanal sayısı söylemedi. Bu hesap tek kanal içindir, iki kanal kayıtta aynı hesap $\approx 10{,}09$ MB verir.

8.4 İşleme ve Eyleyici

Saklanan dizi istenildiği gibi işlenebilir: En büyük elemanı bulunur, sıralanır. Dizinin frekans karşılığı bulunursa en yüksek ve ortalama frekans bileşeni de bulunabilir. Bu bilgiler bu derste görülecek. Gerekirse sonuçlar görüntülenir.

Dışarıya yeniden çıkış verilecekse dönüştürücünün tersini yapan, harekete geçiren bir eleman gerekir: eyleyici (actuator). Örneğin bir yerin sıcaklığını kontrol eden denetleyicide bir hareket gerçekleşir.

> **Ders sonrası not:** Hoca eyleyiciyi "sayısal büyüklüğü analoga dönüştüren" eleman olarak anlattı. Sonradan verdiği tanım daha doğrudur: Eyleyici elektrik sinyalini elektriksel olmayan bir büyüklüğe çevirir. Sayısaldan analoga geçiş büyük olasılıkla sayısal-analog dönüştürücünün (DAC) işidir. Eyleyici ondan çıkan analog sinyali alır.

Özetle bilgisayara veri alınırken 38 derece doğrudan alınmaz. Önce sensörle ölçülür, sonra analog işlemler, ardından sayısal işlemler yapılır. Gerekirse yeniden analog dünyaya dönülür.

8.5 Dönüştürücü ve Enerji

Dönüştürücü, herhangi bir enerji formunu başka bir forma çeviren elemandır. Normalde görevi bir enerji ya da büyüklükten başka birine geçmektir. Barajda suyun kinetik enerjisinin çarkı döndürmesi böyle bir dönüşümdür. Sinyal işlemede ise amaç enerjiyi alıp bilgi üretmektir ve bu bilgi sonunda ayrık bir veri olur. Yani enerji enerjiye değil, bilgiye dönüştürülür.

Dönüştürücü ikiye ayrılır:

- **Giriş dönüştürücüsü (input transducer), sensör:** Elektriksel olmayan enerjiyi elektriksel sinyale çevirir. Örnek: Mikrofon, akustik sesi elektriksel bir ses işaretine çevirir.
- **Çıkış dönüştürücüsü (output transducer), eyleyici:** Elektrik sinyalini elektriksel olmayan büyüklüğe çevirir. Örnek: Hoparlör.

Hoca bu ayrımı slayttaki fizyolojik ölçme sistemleri başlığı altında andı.

8.6 Konuşurken Kaç Kez Kayıp Olur?

Soru: Ben konuşuyorum, sesim analog. Sizin kulağınıza gelen ses de analog. Arada en az kaç kez kayıp olur? **Cevap:** Öğrenci "iki" dedi ve gerekçe olarak mikrofonun alabildiği ve kulaklığın verebildiği ses tonunu gösterdi. Hoca iki sayısını doğru buldu ve kayıpları şöyle açıkladı:

- **Kayıtta:** Mikrofon sesi kaydederken örnekleme ve nicemlemeden kaynaklı hata oluşur.
- **Geri çalmada:** Kulaklıktaki eyleyici dönüştürme yaparken örneklerin arasını doldurmak zorundadır. Sayısal analizde görülen interpolasyon burada devreye girer. Örnekler arasında kaybolan değerler ve nicemleme hataları olduğu için orijinal dalga tam elde edilemez. Bu interpolasyon hatası kulağa yansır.

Sesin kötülüğünü çok hissediyorsanız ya örnekleme ya nicemleme ya da interpolasyon kötü yapılmıştır. Mikrofon ya da eyleyici gibi elemanlar da hatalı olabilir.

9. Analog Sinyalden Sayısal Sinyale

9.1 Analog ve Sayısal Sinyal

Analog sinyal, sonsuz hassasiyette değerler içeren sürekli bir değişkendir. Sürekli değişken denince aklımıza t gelmeli. Zamanın en küçük parçasında dahi bilgi içerir. Bilgisayarda işlenmek için ayrık hale, yani bir diziye dönüştürülür.

Bu dönüşümde kayıp vardır: Ölçmeden kaynaklı kayıp, zamanlamadaki belirsizlik ve ölçmenin kısıtları. Kısıtlara örnek ses kartının ve ekran kartının çözünürlüğüdür. Fotoğraf makinesi de bir ölçme yapar ve sonuçta iki boyutlu bir sinyal elde edilir. Burada kaybın kaynağı cihazın çözünürlüğü olabileceği gibi, makineyi yanlış tutmak ya da hareket ettirmek de olabilir.

Hoca bu etkilerin hepsine nicemleme hatası (quantization error) dendiğini söyledi ve önemli bir kavram olduğunu belirtti.

> **Ders sonrası not:** Hoca blok şemadaki bütün hataları nicemleme hatası diye adlandırdı. Kendisinin bölüm 12.7'de verdiği tanıma göre nicemleme hatası, gerçek değer ile kodlanan değer arasındaki farktır. Yani yalnız genlik yuvarlamasından doğar. Örnekleme, interpolasyon ve sensör hataları büyük olasılıkla ayrı kaynaklar olarak düşünülmelidir.

9.2 Saat Sinyaliyle Örnekleme

Sürekli sinyal, belli aralıklarla örnekleme yapacak bir saat (clock) sinyaliyle çarpılır. Sonuçta merdiven biçiminde bir dalga oluşur. Saat sinyalinin kenarlarından alınan değerler örnekleri verir. Bazı değerler pozitif, bazıları negatif olur. Hoca örneklerin arasındaki aralığın örnekleme sıklığını belirlediğini, örnekleme frekansına f_s diyeceğimizi söyledi.

Bu işlemle kaç eleman saklanacağı belli olur. Bu işlem yapılmasaydı değer ölçme sırasında değişiyor olabilirdi ve yanlış veri kaydedilebilirdi. Sonsuz örnek almak da mümkün değildir. Ama genlik değeri hâlâ bulunmamıştır. Sayısal değere ve 0-1 koduna geçiş sonraki adımlardır.

> **Ders sonrası not:** Hoca burada "aslında tam bir çarpma değil konvolüsyon" dedi. Örnekleme zaman düzleminde çarpmadır. Çarpımın frekans düzlemindeki karşılığı konvolüsyondur. Bu ilişki sayısal bir denemede Python ile doğrulandı: Çarpımın DFT'si ile DFT'lerin dairesel konvolüsyonu arasındaki fark yuvarlama düzeyinde çıktı ($\approx 10^{-15}$). Ayrıca örnekler arasındaki aralık T_s'dir, f_s ise onun tersidir (bkz. bölüm 11.3).

Hoca sürekli sinyalden ayrık sinyale geçişi yeniden özetledi: Önce sinyal vardır, hangi noktalarda örnek alınacağı belirlenir. Örneklerin bir kısmı eğrinin üstünde, bir kısmı altında kalır. Bu işlem sonunda ayrık sinyal 0 ve 1'lerle kodlanacak hale gelir.

9.3 Gösterim: $x(t)$ ve $x[n]$

Soru: Slayttaki iki gösterimden hangisi sayısal işareti gösterir? **Cevap:** Sağdaki, yani n ile gösterilen.

> **Hocanın vurgusu:** Sürekli (analog) işaret t ile ve yuvarlak parantezle gösterilir: $x(t)$. Ayrık (sayısal) işaret n ile ve köşeli parantezle gösterilir: $x[n]$.
>
> **Ders sonrası not:** Hoca "ayrık" ve "sayısal" kelimelerini aynı anlamda kullandı. Kendi anlattığı akışa göre yalnız örneklenmiş sinyalin zamanı ayrıktır ama genliği hâlâ analogdur (bkz. bölüm 11.4, PAM). Sayısal sinyal büyük olasılıkla hem zamanda hem genlikte ayrık olan sinyaldir.

Hoca slayttaki grafik üzerinden önemli bir ayrıntı gösterdi. Sürekli işaret 10 saniye boyunca kaydedilmiştir ve herhangi iki noktası arasında kopukluk yoktur. Ama çizimin kendisi işaretin kendisi değildir. Çizim de örneklenmiş değerlerin arasına interpolasyon yapılarak elde edilmiş, süreklilik görüntüsü veren bir grafiktir.

Aynı slaytta örnekleme ve nicemleme şöyle gösteriliyordu:

- **Zaman ekseni:** 0, 1, 2, 3, 4, … saniyelerde birer örnek alınır.
- **Genlik ekseni:** Genlik ekseni de 2, 3, 4, 5 gibi seviyelere bölünür.
- **Eşleme:** Her örneğin değeri kendisine en yakın seviyeye atanır. 0. saniyedeki örnek sıfıra gelir. 1. saniyedeki örnek iki seviye arasındadır ve yakın olana atanır. Diğerleri de böyledir.
- **Sonuç:** Ayrık işaret merdiven biçiminde çizilir. Slaytta ayrıca 0, 2, 4, 6 biçiminde ölçeklenmiş hali gösterilmişti.

Slaytta ayrık işaret yuvarlak parantezle yazılmıştı. Hoca köşeli parantezle yazılması gerektiğini söyledi.

10. Analog-Sayısal Dönüşümün Dört Adımı ve Filtreleme

Analog-sayısal dönüştürücü (ADC), analog işareti sayısal işarete çevirir. Dört temel adımı vardır:

1. Filtreleme.
2. Örnekleme.
3. Nicemleme.
4. Kodlama (encoding): n bit için oluşan 2^n seviyenin her birine 00…0'dan 11…1'e kadar bir bit dizisi atanır.

10.1 Filtreleme Neden Gerekir?

Bir ses bir sensörle alınırken ortamın gürültüsü de birlikte alınabilir. Hoca kendi bilgisayarının fan sesini örnek gösterdi ve sınıfa sordu. Ses karşıya gitmiyorsa mikrofon ya da kulaklık iyi filtreleme yapıyor demektir. Filtreleme asıl büyüklüğe odaklanır, yan girişimleri (interference) yok eder.

> **Ders sonrası not:** Hoca örnekleme adımını anlatırken işaretin yanındaki küçük dalgalanmaların "bu sampling ile" düzeltildiğini söyledi. Bu temizleme büyük olasılıkla örnekleme değil, kendisinin bir önceki adım olarak anlattığı filtrelemenin etkisidir.

Filtreleme örneklemeden önce yapılmak zorundadır ve yalnız gürültü için değildir. İkinci görevi, işaretin içindeki en yüksek frekansı sınırlamaktır. En yüksek frekans, örnekleme oranını belirler. "Çok örnekleme yapayım, çok kaliteli olsun" düşüncesi gereksizdir. En yüksek frekansa göre örnekleme yaparsanız gereksiz örneklemenin önüne geçersiniz.

Özetle filtreleme üç şey sağlar: gürültüyü ve girişimleri giderir, sinyalin bozulmasını önler ve gereksiz örneklemeyi engeller.

10.2 Frekans Gösteriminde Sinyal

Slaytta yatay ekseni artık zaman değil frekans olan bir grafik gösterildi. Düşük frekans bileşenleri bizim için en çok kullanılan bileşenlerdir. Yüksek frekans bileşenleri bir yerden sonra kesilip atılır. Çünkü bir büyüklüğü en çok tasvir eden faktörler belli bir seviyeye kadarki frekanslardadır. Daha yukarısı ya çok az ilgili ayrıntılar ya da alakasız gürültülerdir. İlgili kısmı almak yeterlidir.

10.3 Şema Üzerinden Akış

Başka bir şemada aynı akış gösterildi: analog sinyal, örnekleme (yalnız zaman ekseni), nicemleme (genlik ekseni), kodlama ve sayısal veri.

11. Örnekleme

11.1 Temel Özellikler

Örneklemenin sonucu ayrık bir sayı kümesi, yani analog sinyalin değerlerini tutan bir dizidir. Örnekler genellikle eşit zaman aralıklarıyla alınır, ama logaritmik gibi farklı aralıklarla yapılanları da vardır.

Örnekleme bir tutma (hold) devresiyle ilişkili olarak yapılır. Bu devre yeterince hızlı olmalıdır. Çünkü kaydedilen veri, örneğin konuşan birinin sesi, sürekli değişir. Veri toplama (data acquisition) sırasında değişimi kaçırmayacak hızda çalışmak gerekir. Örnekleme devresi için ölçülen şeyin ses mi sıcaklık mı olduğu önemli değildir, yalnız ayrık değer üretir.

11.2 Nyquist Ölçütü

Örnek aralıklarından dolayı kayıp oluşur. Bu kaybın geri döndürülebilir olması için bir oran vardır. Bir işaretin $f_1, f_2, \ldots, f_n$ frekans bileşenleri olsun. Önce en yüksek bileşen f_{max} bulunur. Sonra örnekleme oranı şu koşulu sağlamalıdır:

$$f_s > 2f_{max}$$

Örnek: En yüksek frekans bileşeni 1000 Hz ise f_s en az $2 \times 1000 = 2000$ olmalıdır. Hoca bunu "en az 2000 tane örnek, 2000 elemanlık dizi" diye anlattı. Böylece işaret geri döndürülebilir biçimde kaydedilir.

> **Ders sonrası not:** f_s'in birimi örnek/saniye olduğu için 2000 sayısı saniye başına örnek sayısıdır. Kaydın uzunluğu arttıkça dizi de büyür. Bölüm 8.3'teki hesap da bunu gösterir.

Neden iki kat olduğu ispatıyla örnekleme konusunda görülecek. Hoca "iki kata kadar geri döndürmede sorun yaşanır, iki kat ve sonrasında sorun yaşanmaz" dedi.

Ders sonrası not: Slayttaki koşul kesin büyüktür, tam $2f$ sınırı güvenli değildir. Python ile hesaplandı: $\sin(2\pi ft)$ tam $f_s = 2f$ ile örneklenirse, faz sıfırken bütün örnekler sıfır çıkar. Örnekler $\cos(2\pi ft)$ gibi tepelerden geçerse $+1, -1, +1, \ldots$ çıkar. Yani tam $2f$'de sonuç fazına bağlıdır.

11.3 Örnekleme Periyodu

Hoca f_s bilindiğinde örnekler arasındaki zaman aralığının nasıl bulunacağını sordu, çünkü örnekleme zaman düzleminde yapılır. Sınıftan doğru cevap geldi:

$$T_s = \frac{1}{f_s}$$

T_s'ye örnekleme periyodu ya da örnekleme aralığı (sampling interval) denir, buradaki s "sampling" demektir. Hoca bunu, işaretin f_s tane eşit aralığa bölünmesi olarak açıkladı. Bu bölme bir saniyelik süre içindir (ara adım eklendi). Periyot ile frekansın çarpımı 1'dir, dolayısıyla $T_s \cdot f_s = 1$. Bu formül çok kullanılacaktır.

11.4 Örnekleme Yöntemleri ve PAM

Üç temel yöntem vardır. Hoca bunların ayrıntısının daha çok sinyal işleme konusu olduğunu söyledi.

- **İdeal örnekleme (ideal sampling):** Her örnekleme anında bir impuls sinyali gönderilir.
- **Doğal örnekleme (natural sampling):** Belirli genişlikte darbeler kullanılır. Hoca bir değer alınıp sonraki örneğe kadar aranın doldurulduğunu söyledi.
- **Düz tepeli örnekleme (flat-top):** Bir örnekleme anından sonrakine kadar kare dalga biçiminde tek bir genlik tutulur. Hoca bunun çok kullanılmadığını, "o anlık al, kaydet" biçiminde olduğunu söyledi.

  Ders sonrası not: Hoca ideal örneklemeyi "dip dibe" impulslar ve "T_s'in çok küçük olduğu" durum olarak anlattı. Oysa kendi tanımına göre impulslar örnekleme anlarında gönderilir ve aralarında T_s vardır. Sıfıra giden şey büyük olasılıkla T_s değil, impulsun genişliğidir. Doğal örneklemede de darbenin üstü büyük olasılıkla o süre boyunca sinyalin kendisini izler. Tutulan sabit değer düz tepeli örneklemeye aittir.

Zaman ekseninde alınan bu örneklerin yüksekliğini bulma işlemine darbe genlik modülasyonu (Pulse Amplitude Modulation, PAM) denir. Sonuç hâlâ analog değerlerden oluşur, çünkü genlik henüz nicemlenmemiştir.

11.5 Örnekleme Frekansının Etkisi

Hoca frekansı f olan bir sinüzoidin farklı oranlarla örneklenmesini slayt üzerinden gösterdi.

- $f_s = 2f$**:** Bir periyotta iki örnek alınır. Örnekler birleştirilince sinüzoide yakın bir şekil çıkar. Hoca önce dört örnek saydı, bir öğrencinin uyarısıyla düzeltti: Üstteki durumda iki örnek vardır.
- $f_s = 4f$**:** Bir periyotta dört örnek alınır. Birleştirme hatası biraz daha azdır.
- $f_s = f$**:** Bir periyotta bir örnek alınır. Araları birleştirince sonuç bozulur.

  Ders sonrası not: Python ile hesaplandı: Bir sinüs tam $f_s = f$ ile örneklenirse bütün örnekler aynı değeri alır, yani birleştirince düz bir çizgi çıkar. Değer yalnız faza bağlıdır. Sinüzoidin salınımı tamamen kaybolur. $f_s = 4f$ ve faz sıfırken örnekler $0, 1, 0, -1, \ldots$ olur. Hoca bu örnekte "işaret f kadar periyotta kendini tekrar ediyor" dedi. Bölüm 11.3'e göre periyot $1/f$'dir.

Terimler şöyledir:

- **Aşırı örnekleme (oversampling):** $f_s > 2f$.
- **Nyquist oranında örnekleme (Nyquist rate sampling):** $f_s = 2f$.
- **Yetersiz örnekleme (undersampling):** $f_s < 2f$.

> **Ek bilgi (derste anlatılmadı):** Yetersiz örneklemede oluşan bozulma büyük olasılıkla, hocanın dersin başında bölüm adı olarak andığı örtüşmedir (aliasing). Yüksek frekanslı bileşen, örneklerde daha düşük bir frekans gibi görünür.

Hoca farklı bir örnek olarak örnekleme frekansı 1024 Hz alınmış bir osiloskop slaytı gösterdi. Değişken önce sayısal hale getirilir, sonra nicemleme ile 0 ve 1'lerle kodlanır.

11.6 Örnekleme Teoremi

Teoreme göre sinyal, içindeki en yüksek frekansın en az iki katı oranda örneklenirse geri elde edilebilir (reconstruction). Hoca teoremi Nyquist'e atfetti ve ayrıntısının sonraya kaldığını söyledi.

Mantığı şudur: İşaret frekans düzlemine geçirilir, en yüksek bileşeni bulunur. f_{max}'ın altındaki bütün frekansları almak (alçak geçiren, low-pass) sinyali bütün bileşenleriyle almak demektir. Fakat belli bir aralıktaki frekanslar da alınabilir: Bileşen en küçük ve en büyük sınır arasındaysa alınır, dışındaysa alınmaz. Alınmayan kısım bastırma (suppression) ile aradan çıkarılır.

> **Ders sonrası not:** Hoca bastırmayı "kaydetmediğimiz örnekler" üzerinden anlattı. Kendi anlatımındaki ölçüt frekanstır, yani bastırılan şey büyük olasılıkla zaman örnekleri değil frekans bileşenleridir.

11.7 Hareket Bulanıklığı Örneği

Bastırmanın önemini hoca görüntüden bir örnekle anlattı. Telefonla fotoğraf çekerken makine biraz hareket ederse görüntüde kayma oluşur: hareket bulanıklığı (motion blur). Bunu geri alan bulanıklık giderme (de-blurring) algoritmaları vardır. Önce işaretten kayma miktarını gösteren frekans bilgisi bulunur. Sonrası kolaydır: O miktar kadar telafi (compensation) yapılarak kayma geri alınır.

12. Nicemleme

12.1 Bölgeler ve Adım Büyüklüğü

Nicemleme, örneklemeden elde edilen değerlerin kodlanmasını sağlar. İşaret zaman düzleminde olduğu gibi genlik düzleminde de sonsuz değer içerir ve bunu sonlu hale getirmek gerekir. Bunun için işaretin en küçük ve en büyük sınırına bakılır. Genlik ekseni, bit sayısına göre L tane bölgeye (zone) ayrılır. Bölgelerin genişliği eşittir ve bu genişliğe Δ denir:

$$\Delta = \frac{V_{max} - V_{min}}{L}, \qquad L = 2^n$$

12.2 Örnek: 24 V ile -12 V Arası, 2 Bit

Hoca değerleri tahtada kurdu. Önce $V_{min} = -14$ V ve $n = 4$ dedi, sonra sonucun tam sayı çıkması için $V_{min} = -12$ V ve $n = 2$ olarak değiştirdi.

- **Verilen:** $V_{max} = 24$ V, $V_{min} = -12$ V, $n = 2$ bit.
- **Aralık:** $24 - (-12) = 36$ V.
- **Bölge sayısı:** $L = 2^2 = 4$.
- **Adım:** $\Delta = 36/4 = 9$ V.

Bölgeler en alttan başlayarak 9'ar volt eklenerek bulunur:

Bölge	Aralık (V)	Kod
Birinci	$[-12,\ -3)$	00
İkinci	$[-3,\ 6)$	01
Üçüncü	$[6,\ 15)$	10
Dördüncü	$[15,\ 24]$	11

Hoca, bölgelerin çakışmaması için sonraki bölgeyi "$-3{,}01$", "6 virgül küsür" diye başlattı. Yukarıdaki yarı açık aralıklar aynı şeyi söyler.

Soru: 18 V hangi bölgeye düşer, nasıl saklanır? **Cevap:** $15 \le 18 \le 24$ olduğu için son bölgeye düşer ve 11 ile saklanır. 2 bit ile yalnız 00, 01, 10 ve 11 yazılabilir, başka kod yoktur.

12.3 Bölge Numaraları

Bölgeler 1'den başlayarak sayılabildiği gibi 0'dan $L-1$'e kadar da numaralanabilir. Sıfırdan başlamak ikili açılımdan gelir: Kodun onluk karşılığı doğrudan bölge numarasıdır. Üç bit ile sekiz durum yazılır:

Kod	000	001	010	011	100	101	110	111
Bölge numarası	0	1	2	3	4	5	6	7

12.4 Örnek: -20 V ile +20 V Arası, 8 Bölge

- **Verilen:** $V_{min} = -20$ V, $V_{max} = 20$ V, $L = 8$.
- **Adım:** $\Delta = 40/8 = 5$ V.
- **Gerekli bit:** $2^n = 8$ olduğundan $n = 3$. Sınıftan doğru cevap geldi.

Slayttaki kodlama şöyledir (hoca ilk ikisini okudu, diğerleri aynı kuralla gelir):

Kod	Aralık (V)	Kod	Aralık (V)
000	$[-20,\ -15)$	100	$[0,\ 5)$
001	$[-15,\ -10)$	101	$[5,\ 10)$
010	$[-10,\ -5)$	110	$[10,\ 15)$
011	$[-5,\ 0)$	111	$[15,\ 20]$

Soru: Bu sinyal bellekte ne kadar yer kaplar? **Cevap:** Sınıftan iki yanlış cevap geldi. İlki 15 bitti, yani 5×3. Hoca düzeltti: 5, ihtimal sayısı değil, her bölgenin genişliğidir. İkincisi 24 bitti, yani 8×3. Bu da yanlıştır, çünkü bir örnek 8 bölgenin hepsinde değil yalnız birinde olur. Doğru cevap yine sınıftan geldi:

$$\text{Bellek} = N \times 3 \text{ bit}$$

Burada N örnek sayısıdır. Soruda örnek sayısı verilmediği için sonuç sayıyla söylenemez. Her örnek için bölgesi bulunur ve 3 bit ile yazılır. Sonraki örnek de 3 bittir.

> **Hocanın vurgusu:** Bu bazen şaşırtıcı bir sorudur. Bölge sayısı ve bölge genişliği belleği doğrudan vermez. Bit sayısı bölge sayısından gelir, toplam bellek de örnek sayısıyla bit sayısının çarpımıdır.

12.5 Orta Nokta Kuralı

Hoca burada bir kararsızlığa dikkat çekti. Bir örneğin iki kod arasında kaldığını düşünelim. Bir sınır 001, üstteki sınır 010 kodlu olsun. Değerler 5 V ve 14 V ise değer hangi koda yazılacak? En basit yaklaşım orta noktaya bakmaktır: Değer orta noktadan büyükse üst sınırın koduyla, küçükse alt sınırın koduyla kodlanır. Bu örnekte orta nokta $(5+14)/2 = 9{,}5$ V'tur (ara adım eklendi).

Hoca ardından $\Delta = 5$ V'luk örneğe dönüp 17 V'u sordu ve şu sonuca ulaştı: 17 V, 15 ile 20 arasındaki bölgededir, orta nokta 17,5 V'un altında olduğu için alt sınırla yazılır, kodu 110'dur. 16 V ve 15 V de 110, 17,5 V ve üzeri 111'dir.

> **Ders sonrası not:** Bu sonuç aynı örnekteki slayt kodlamasıyla çelişir. Python ile kontrol edildi: Bölüm 12.4'teki tabloya göre 15, 16, 17 ve 17,5 V'un hepsi $[15, 20]$ bölgesindedir ve kodu 111'dir. 110 kodu $[10, 15)$ bölgesine aittir. 17,5 V ise bu bölgenin orta noktasıdır. Orta nokta kuralı ise kodların bölgelere değil sınır değerlere

verildiği farklı bir düzendir. Bu örnekte $-20, -15, \ldots, 20$ olmak üzere 9 sınır değeri vardır ve bunlar 3 bitlik 8 koda bire bir sığmaz. Bu yüzden bölge düzeninde 17 V'un kodu 111 alınmalıdır.

12.6 Gerekli Bit Sayısı

$2^n = 8$ ise $n = 3$ olduğunu üstel olarak bulduk. Aynı iş logaritmayla yapılır ve gereken en az bit sayısını verir. 8 için kolaydır, ama 88 olursa logaritma gerekir:

- **Verilen:** 88 durum.
- **İşlem:** $\log_2 88 \approx 6{,}46$.
- **Yorum:** 6 bit yalnız $2^6 = 64$ durum gösterir, 88'i saklamaz. Üste yuvarlanır.
- **Sonuç:** 7 bit, yani $2^7 = 128$ durum.

12.7 Normalizasyonlu Nicemleme Örneği ve Nicemleme Hatası

Slaytta eksi ve artı değerler alan, örneklenmiş bir sinyal vardı. 8 durum olduğu için kodlar 0'dan 7'ye gider. Tablo her örnek için normalize değeri, nicemleme seviyesini, hatayı ve kodu veriyordu. Hoca iki satırı okudu.

Örnek	Normalize değer	Nicemleme seviyesi	Hata	Kod (onluk)	Kod (ikili)
$-6{,}1$	$-1{,}22$	$-1{,}5$	$0{,}28$ (büyüklük)	2	010
okunmuyor	$1{,}5$	$1{,}5$	0	5	101

Tablodan okunmayan ara adımlar şöyle kurulur (ara adım eklendi):

- **Normalizasyon katsayısı:** $-6{,}1 / -1{,}22 = 5$.
- **Hata:** $|-1{,}22 - (-1{,}5)| = 0{,}28$.
- **Kod ile seviye:** Kod 5 için seviye $1{,}5$ ise bir kod farkı 1 seviye eder. Kod 2 için seviye $1{,}5 - 3 = -1{,}5$ çıkar. Bu, altyazıdaki bozuk değerle uyumludur.

Hoca normalizasyonun en küçük ve en büyük değer aralığını belli bir aralığa çektiğini, her değeri normalize etmenin de gerekmediğini, bunun bir anlamda bit sayısını düşürmek için kullanıldığını söyledi.

> **Ders sonrası not:** Bu örnekte kod bit sayısı $\log_2 8 = 3$ olarak bölge sayısından gelir. Normalizasyon bölge sayısını değiştirmediği için bu tablodaki kod uzunluğunu düşürmez.

Nicemleme hatası (quantization error): Olması gereken değer ile kodlanan değer arasındaki farktır. Değer belli bir seviyeye yuvarlandığı için oluşur. Bölge sayısı artarsa hata azalır, azalırsa artar.

Peki bölge sayısını artırmak her zaman iyi midir? Hayır. Hatayı azaltmak daha fazla bellek, yani daha fazla bit hızı (bitrate) ister. Aradaki denge gözetilmelidir.

12.8 Örnek: 12 Bitlik ADC

Slayttaki soru: Giriş aralığı 0-10 V olan 12 bitlik bir analog-sayısal dönüştürücünün doğruluğu (accuracy) nedir?

- **Aralık:** $10 - 0 = 10$ V.
- **Seviye sayısı:** $2^{12} = 4096$.
- **Adım:** $\Delta = 10/4096 \approx 0{,}00244$ V.

Hoca sonucu "nicemleme hatası $\pm 0{,}0024$ V" olarak verdi.

Ders sonrası not: Python ile hesaplandı: $\Delta = 0{,}00244140625$ V. Bu değer 1 LSB, yani bir adımdır. Slayt, altyazıda bozuk geçen ifadeye göre doğruluğu en düşük anlamlı bit (LSB) cinsinden veriyor olabilir. Bu durumda ± 1 LSB için $\pm 0{,}0024$ V tutarlıdır. Ama nicemleme hatası olarak bakılırsa, en yakın seviyeye yuvarlamada en büyük hata yarım adımdır: $\pm\Delta/2 \approx \pm 0{,}00122$ V.

13. Sayısallaştırma ve Geri Oluşturma Zinciri

Hoca örnekleme ile ilgili bir sonraki slaytları, ileride tekrar edileceği için geçti ve zinciri özetledi.

Sayısallaştırma (analogdan sayısala):

1. Bant sınırlama: Her frekans alınmaz, belli bir aralık alınır. Bu hem örnekleme hem gürültü giderme için yapılır.
2. Örnekleme.
3. Nicemleme.
4. Kodlama.

Geri oluşturma (reconstruction, sayısaldan analoga):

1. Sayısal-analog dönüştürücü (Digital to Analog Converter).
2. Hocanın yine "sampling" dediği adım: Sayısaldan sürekliye geçerken yapılan interpolasyonun ön adımıdır.
3. İnterpolasyon ve görüntü bastırma (image rejection): İki örnek arası birleştirilir.

14. Verinin ve Sayıların Bilgisayarda Gösterimi

Hoca bu kısmın çoğunun bilgisayar bilimlerine girişten bilindiğini söyleyerek hızlıca geçti.

14.1 Bit, Bayt ve Bir Sinyal İşleme Örneği

Bit en küçük veri miktarıdır: doğru ya da yanlış. Bitlerden daha büyük birimler kurulur:

- **Nibble:** 4 bit.
- **Bayt (byte):** 8 bit, yani 2 nibble.
- **Word:** 16 bit.

Bunlardan yola çıkarak işaretli ve işaretsiz tam sayı (signed, unsigned integer), uzun tam sayı (long integer) ve ikiye tümleyen (two's complement) kavramları oluşur.

Slayttaki canlandırmada ses, analog-sayısal dönüştürücü ile sayıya çevrilip bir sayısal sinyal işlemciye (DSP) gidiyor. DSP işareti işliyor, sayısal-analog dönüştürücü de sonucu dışarı veriyor.

14.2 Kapasite ve Hız Birimleri

Bilgisayardaki birimler 10'un ve 2'nin kuvvetlerinden oluşur. On tabanı kilo, mega gibi önekleri, 2 tabanı bit ve baytı verir. Önekler kilo (K), mega (M), giga (G), tera (T) ve peta (P) diye gider. Önemli olan önek değil, ondan sonra gelen birimdir: bit mi, bayt mı, hız mı, frekans mı?

- **Hertz:** Frekans birimidir. Saniye başına saat çevrimi (clock cycle), yani titreşim sayısını söyler. İşlemci hızları megahertz ya da gigahertz ile ifade edilir.
- **Bayt:** Depolama birimidir. RAM ve sabit diskte kilobayt, megabayt, gigabayt ve terabayt kullanılır.

Hoca birimin önemini amper ile gösterdi: 1 amper çok yüksek bir değerdir, 1 volt ise o kadar değildir. İnsana asıl çarpan amperdir. Çok düşük voltta yüksek amper çok daha fazla etki yapabilir. Küçük birimler tarafında mili, mikro, nano, piko ve femto vardır. Saniye için 10^{-3}, 10^{-6} diye gider. Aynı önekler nanosaniye gibi zamanda, nanometre gibi uzunlukta kullanılır.

14.3 Veri Tipleri

Programlamada veri tiplerini soyutlama için kullanırız.

- **Sayılar:** İşaretli, işaretsiz, tam sayı, kayan noktalı, karmaşık, rasyonel ve irrasyonel olabilir.
- **Metin:** Karakterlerden ve karakter dizilerinden oluşur.
- **Ses:** Bir dizidir.
- **Görüntü:** İki boyutlu bir matristir.
- **Video:** Üç boyutludur, yani birçok matrisin ardışık dizisidir.
- **Mantıksal veri:** Doğru ya da yanlış.
- **Komutlar (instructions).**

14.4 Konumsal Sayı Sistemleri

Konumsal sayı sistemlerinde taban (radix) R ise rakamlar 0'dan $R-1$'e kadardır. Onluk sistemde 0'dan 9'a kadardır, 10 dahil değildir. Rakamların birleşimi sayıyı verir. Tam sayıya kesir eklenecekse kesir noktası konur. Noktanın solunda $R^0, R^1, R^2, ...$, sağında $R^{-1}, R^{-2}, ...$ kuvvetleri vardır. Bu, sayma sistemlerinin genel modelidir ve ikilik sistem için de geçerlidir.

Onluk sistemde hoca slayttaki sayıyı basamaklarına ayırdı: Üç binlik, yüzlükler, onluklar ve birlikler. Negatif sayı için başa eksi konur ve bu yön bilgisi taşır.

$$3446 = 3 \cdot 10^3 + 4 \cdot 10^2 + 4 \cdot 10^1 + 6 \cdot 10^0$$

İkilik sistemde basamak değerleri birler, ikiler, dörtler, sekizler ve on altılar diye gider. ABC gibi üç basamaklı bir sayıda A tane dörtlük, B tane ikilik ve C tane birlik vardır.

14.5 İşaretsiz ve İşaretli Sayılar

n bit ile işaretsiz en büyük değer $2^n - 1$'dir. Slaytta 3, 5 ve 8 bitlik tablolar yan yana duruyordu: Küçük değerler hepsinde aynı görünür. Ama 3 bit ile 7'den öteye gidilemez, daha fazla bit daha ileri götürür. Hocanın örneği 10 bitti: $2^{10} - 1 = 1023$, yani 0 ile 1023 dahil arası.

Negatif sayı gerekiyorsa bir bit işarete gider. 10 bitin 9 biti kalır. Üç yöntem vardır: işaret-büyüklük (sign magnitude), bire tümleyen (one's complement) ve ikiye tümleyen (two's complement). İkiye tümleyende aralık şudur:

$$-2^{n-1} \quad \text{ile} \quad 2^{n-1} - 1$$

Örnek: 5 bit ile bir bit işarete gider. Aralık $-2^4 = -16$ ile $2^4 - 1 = +15$ arasıdır. Bu değerler Python ile doğrulandı: -16 için 10000, $+15$ için 01111.

İkiye tümleyen matematiksel işlemlerde hatasız sonuç üretir ve verimlidir (efficient). Sıfır bu gösterimde bir kez yazılır. Bire tümleyen ve işaret-büyüklükte $+0$ ve -0 belirsizliği vardır ve hatalı sonuç verebilirler. Bu yüzden çok kullanılmazlar. Ayrıntılar lojik dersinin konusudur.

14.6 Gerçel Sayılar

Sayıların çoğu tam sayı değildir, bu yüzden tam sayı gösterimi yetmez. Virgüllü sayılar gösterilemez, aralık sorunu vardır, hassasiyet (precision) veride hataya yol açabilir. Gerçel sayılar kayan noktalı (floating point) gösterimle saklanır. Hoca bu derste bununla çok uğraşılmayacağını, istenirse bir ara anlatılabileceğini söyledi.

> **Ders sonrası not:** Hoca standardın adını önce "IEEE 744" dedi, hemen sonra doğrusunu söyledi: IEEE 754. Standardın iki gösteriminden yalnız 32 bitlik olanı altyazıda geçiyor. Diğeri büyük olasılıkla 64 bittir.

Soru: Bir öğrenci bu standardı görmediğini söyledi. **Cevap:** Hoca bir ara anlatabileceğini söyledi. Öğrenci kendisinin de araştırabileceğini ekledi. Hoca bunun başka derslerde gerekebilecek bir genel kültür olduğunu belirtti.

14.7 Gerekli Bit Sayısı ve Renk Örneği

Görüntü işlemede renkler kodlanır. Renk sayısına uygun bit sayısı seçilir. 20 renk için 4 bit yetmez, çünkü $2^4 = 16$'dır. En az 5 bit gerekir, $2^5 = 32$. Genel formül:

$$\text{Gerekli bit sayısı} = \lceil \log_2(\text{durum sayısı}) \rceil$$

Buradaki tavan (ceil) fonksiyonu üste yuvarlamadır.

> **Hocanın vurgusu:** Logaritmanın sonucu tam sayı değilse üste yuvarlanır. Hoca bu tavan işlemini örnekler üzerinden tekrar tekrar gösterdi.

- **68:** $\log_2 68 \approx 6{,}09$, üste yuvarlanınca 7 bit. Hoca önce "iki virgül küsür" dedi, hemen altı virgül küsür diye düzeltti.
- **9:** $\log_2 9 \approx 3{,}17$, üste yuvarlanınca 4 bit.

Bu değerler Python ile doğrulandı. Hoca, bulunan bit sayısıyla bütün dizilimlerin yazıldığını, burada sıralamanın önemli olduğunu, yani kombinasyona değil permütasyona bakıldığını söyledi.

> **Ders sonrası not:** Hoca formülü anlatırken büyüklüğün "en yüksek değerine, aralığına" bakılacağını söyledi. Örneklerdeki hesap durum sayısı üzerinden yapılır. 0'dan başlayan bir büyüklükte durum sayısı, en yüksek değerin bir fazlasıdır. Bölüm 14.5'teki 10 bit örneğinde en yüksek değer 1023, durum sayısı 2^{10}'dur.

14.8 Dönüşüm ve Kodlama

Slayttaki sonraki başlıklar BCD (binary coded decimal), sayı sistemleri arası dönüşüm, karşıya gönderilen bilginin hata denetimi, ASCII ve Unicode kodları ile NOT, OR, AND gibi mantıksal işlemlerdi. Hoca bunların ayrık matematik, lojik ve assembly derslerinde görüldüğünü, bu dersle doğrudan ilgili olmadığını söyledi. ASCII'de bir karakter tablosundaki her karakterin belli bir sistematikle kodlanmış sayısal karşılığı vardır. Unicode ise ASCII'den daha ayrıntılıdır.

> **Hocanın vurgusu:** Dönüşüm (conversion) ile kodlama (coding) birbirine karıştırılmamalıdır.

Onluk bir sayının ikilik sayıya dönüşümü, sayının doğrudan matematiksel karşılığını verir. İkilik kodla kodlanması ise doğrudan karşılığı değildir, algoritmik bir işlemdir. Slayttaki örnekte her rakam ayrı ayrı ele alınıyordu: "biri böyle almış, üçü böyle almış". Matematikte böyle bir işlem yoktur.

- **Dönüşüm:** $13 \rightarrow 1101$.
- **BCD kodlama:** $13 \rightarrow 0001\ 0011$.

Dersin Toparlaması

Bu derste önce dersin yeri görüldü: Sinyaller ve sistemler, devre teorisi ile bilgisayar bilimlerinden gelen bilgiyi sayısal işaret işlemeye, görüntü işlemeye ve yapay zekaya taşıyan bir geçiş dersidir. Ders sürekli ve ayrık olmak üzere iki koldan ilerleyecek, konvolüsyon, denklemler ve Fourier, Laplace, z dönüşümleri üzerinden sinyaller farklı düzlemlerde incelenecek. Veri, bilgi, sistem ve sinyal kavramlarıyla bunun zemini kuruldu.

Asıl teknik çekirdek ölçme zinciridir: fiziksel büyüklük, dönüştürücü, analog işleme, filtreleme, örnekleme, nicemleme, kodlama, saklama, işleme ve gerekirse sayısal-analog dönüşüm ile eyleyici. Analog sinyal zamanda örnekleme ile, genlikte nicemleme ile sonlu hale gelir. Örnekleme için $f_s > 2f_{max}$ ve $T_s = 1/f_s$; nicemleme için $\Delta = (V_{max} - V_{min})/2^n$; bit sayısı için $\lceil \log_2(\text{durum sayısı}) \rceil$; bellek için örnek sayısı çarpı bit sayısı akılda tutulmalıdır. Son olarak sayıların bilgisayardaki gösterimi hatırlatıldı. Sonraki derste sinyal ve sistemin ne olduğuyla dersin asıl içeriğine girilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.